Updated 90-Day Career Switch Roadmap

> From Enterprise / Production Experience to AI-Powered Backend & GenAI Engineering

Target: 18 LPA+ | **Timeline:** 12 weeks / approximately 90 days

Core Direction: Python + Backend + GenAI + Cloud + System Design + DSA

> **Target Profile:** Software Engineer with enterprise production experience who builds AI-powered systems.

1. Career Strategy

Do not restart from zero as a traditional ML Engineer. Build on enterprise production, SQL, debugging, microservices and data-platform experience, then add Python backend development, cloud and GenAI.

Transition path: Enterprise / Production Experience → Python Backend Engineering → AI-Powered Backend Engineering → GenAI Applications.

Target roles: AI / GenAI Engineer, AI Application Engineer, LLM Engineer, Backend Engineer + AI/GenAI, Python Backend Developer, Applied AI Engineer, AI Platform Engineer and Software Engineer – GenAI.

2. Priority System

Priority 1 — Projects and real skills: strongest evidence of your transition.

Priority 2 — Python + Backend: become comfortable writing production-oriented code and APIs.

Priority 3 — GenAI Engineering: LLMs, RAG, embeddings, tool calling, agents, evaluation and production AI systems.

Priority 4 — DSA: pattern recognition and interview problem solving, not competitive programming.

Priority 5 — Cloud: practical AWS knowledge and deployment.

Priority 6 — Certifications: support learning and credibility; never replace projects.

3. Daily Schedule

Monday–Friday: 2.5–3 hours
45–60 min DSA
60 min main learning topic
45–60 min project/coding.

Busy-day minimum: 90 minutes. Do not abandon the entire week because one day was busy.

Saturday: 5–6 hours — 1.5–2h DSA, 2h project, 1h learning/certification, 30–60m revision.

Sunday: 4–5 hours — project work, weekly revision, DSA review, GitHub cleanup, and later in the roadmap resume/LinkedIn/job research.

PHASE 1 — FIRST 3 WEEKS

Main objective: Build Python coding confidence, OOP fundamentals, core data structures, DSA problem-solving ability and pattern recognition.

Do not rush into trees, graphs and DP before you are comfortable translating ideas into Python code.

WEEK 1 — Python Foundations + Arrays / Strings / Hashmaps

Day 1: Variables, data types, lists, tuples, sets, dictionaries, loops, conditionals and functions. Learn Big-O basics: O(1), O(n), O(n^2), O(log n). Start simple array traversal/counting/max-min problems.

Day 2: Lists, indexing, slicing, append/remove/sort. Practice traversal, frequency counting, duplicate detection and basic array manipulation.

Day 3: Strings: indexing, slicing, string methods, character iteration and join. Practice reverse string, palindrome and frequency problems.

Day 4: Dictionaries: creation, access, update, iteration and .get(). Learn hashmap lookup and frequency tracking. Practice Two Sum-style problems.

Day 5: Sets and membership lookup. Practice duplicate detection, intersections and missing-element problems.

Day 6: Review arrays, strings, dictionaries and sets. Solve 5–6 mixed Easy problems.

Day 7: Weekly revision. Rewrite 2–3 solutions from memory and explain solutions aloud.

Week 1 target: Approximately 15–20 quality problems. Focus on understanding input → processing → output.

WEEK 2 — Python OOP + Core DSA Patterns

Day 8: Classes, objects, __init__, instance variables and methods. Build a simple Ticket class.

Day 9: Class variables, instance variables, @classmethod and @staticmethod. Extend the Ticket example.

Day 10: Two pointers: same-direction and opposite-direction patterns. Practice palindrome, sorted-array and pair problems.

Day 11: Sliding window: fixed-size and variable-size windows. Practice maximum-sum and longest-substring style problems.

Day 12: Prefix sums and range-sum concepts. Introduce hashmap + prefix sum.

Day 13: Stack using Python lists. Practice Valid Parentheses and other basic stack problems.

Day 14: Review OOP, two pointers, sliding window, prefix sums and stack. Solve 5–6 mixed problems.

Week 2 target: Approximately 30–35 total quality problems. Start asking: “Which pattern does this problem belong to?”

WEEK 3 — Linked Lists + Binary Search + Python Data Structures

Day 15: Linked list fundamentals: Node, next, traversal. Implement your own Node and LinkedList with insert/delete/search/traversal.

Day 16: Linked list problems: reverse linked list, middle node and cycle-detection concepts.

Day 17: More linked lists: merge lists, remove node, fast and slow pointers.

Day 18: Binary search fundamentals: boundaries, left/right pointers and mid calculation.

Day 19: Recursion: base case, recursive case and call stack. Practice simple recursive problems.

Day 20: Python collections: deque, defaultdict, Counter and an introduction to heapq.

Day 21: Full revision of every pattern covered. Solve 5–8 problems without notes, then review mistakes.

End-of-Week-3 target: 40–45 quality problems and confidence with arrays, hashing, two pointers, sliding window, stacks, linked lists, binary search and basic recursion.

PHASE 2 — WEEKS 4–6: Backend + Data + GenAI Foundations

Week 4: HTTP, REST, JSON, status codes, authentication/JWT, pagination, validation, FastAPI, Pydantic, dependency injection, middleware, async endpoints and OpenAPI.

Mini Project #1: Production Ticket Management API using Python, FastAPI, PostgreSQL, authentication, logging, validation, error handling and Docker.

DSA: Continue linked lists, binary search, stack/queue and begin trees.

Week 5: PostgreSQL joins, indexes, transactions, ACID, normalization, isolation and query plans. Learn Redis caching, TTL, sessions and rate limiting.

LLM fundamentals: tokens, context windows, temperature, system prompts, structured outputs, hallucinations, model selection and function/tool calling.

Mini Project #2: AI Ticket Classifier producing category, severity, service, summary and recommended action.

Week 6: RAG: ingestion, chunking, embeddings, vector search, metadata, top-k retrieval, context construction and reranking. Use PostgreSQL + pgvector.

Project: Enterprise Knowledge Assistant for runbooks and engineering documentation with source-backed retrieval.

Certification Track

Maximum certifications during the 90 days: 2.

Certification 1 — Practical GenAI / AI credential: Start around Weeks 5–7. It should support LLM applications, prompting, embeddings, RAG and AI application engineering. Limit it to 2–3 hours per week.

Certification 2 — AWS: Start preparation around Weeks 8–12 after practical exposure to IAM, EC2, S3, RDS, Lambda and CloudWatch. Limit it to 2–3 hours per week.

Rule: Before spending 10 hours on certification content, ask whether those 10 hours would improve your flagship project more. If yes, prioritize the project.

Do not delay job applications, projects or interview preparation to collect certificates.

PHASE 3 — WEEKS 7–8: Tool Calling + Agents + Cloud

Week 7: Function/tool calling. Learn how an LLM selects backend tools, receives results and interprets them. Example tools: get_service_health(), get_failed_transactions(), get_recent_deployments(), search_runbooks() and get_previous_incidents().

Week 8: Agent workflows: state, workflows, tool use, retries, guardrails, human-in-the-loop and structured outputs. LangGraph may be used, but prioritize one reliable workflow over complicated multi-agent demos.

AWS: IAM, EC2, S3, RDS, Lambda and CloudWatch at a practical level.

DSA: Trees, BFS, DFS, graphs, heaps and introductory backtracking. Target 75–80 total quality problems.

Career: Begin job-market research and collect recurring requirements from job descriptions.

PHASE 4 — WEEKS 9–10: Flagship Portfolio Project

Project: AI Production Incident Investigator.

Architecture: User → FastAPI → AI Orchestrator → SQL Tool / RAG Tool / API Tool → PostgreSQL / pgvector / Service APIs → LLM → Incident Report.

Example: “Why did payment failures increase between 2–3 PM?”

The system should query data, detect error spikes, retrieve runbooks, check deployments, compare previous incidents, correlate evidence and generate a likely root cause.

Output: Incident, Impact, Likely Root Cause, Evidence, Confidence and Recommended Actions.

Natural-language-to-SQL safety: use a read-only database, allowed tables/columns, validation and query limits. Never blindly execute arbitrary LLM-generated SQL.

Tech: Python, FastAPI, PostgreSQL, pgvector, LLM API, RAG, tool calling, workflow/agent logic, Docker and AWS.

DSA: Mixed revision and Medium problems; begin basic DP. Target 90–100 quality problems.

Jobs: Start applying actively; do not wait for Day 90 or perfect certification completion.

PHASE 5 — WEEKS 11–12: Interview Mode

DSA revision: Arrays, strings, hashmaps, two pointers, sliding window, prefix sums, stack/queue, linked lists, binary search, trees, heaps, graphs, recursion, backtracking and basic DP.

System design: URL shortener, rate limiter, notification system, file upload service and chat system.

AI system design: RAG architecture, ingestion, chunking, embeddings, retrieval, reranking, latency, cost, caching, monitoring, evaluation, security and prompt injection.

Interview questions: What is RAG? Why RAG instead of fine-tuning? How do embeddings work? How do you reduce hallucinations? How do you evaluate an LLM app? How do you defend against prompt injection? How do you safely execute LLM-generated SQL? How do you reduce latency/cost? How would you handle one million documents? How would you monitor a production AI application?

Final DSA Target

Target: Approximately 100 quality problems.

Arrays / Strings / Hashing: 25–30.

Two Pointers / Sliding Window / Prefix Sum: 15–20.

Stack / Queue: 8–10.

Linked Lists: 8–10.

Binary Search: 8–10.

Trees / Heaps: 15–20.

Graphs: 8–10.

Recursion / Backtracking: 5–8.

Basic DP: 5–8.

Problems can overlap categories. The goal is pattern recognition, explanation and clean implementation.

Portfolio by Day 90

Project 1: Production Ticket Management API — Python + FastAPI + PostgreSQL + authentication + Docker.

Project 2: Enterprise RAG Assistant — Python + FastAPI + PostgreSQL + pgvector + embeddings + LLM.

Project 3 — Flagship: AI Production Incident Investigator — Python + FastAPI + PostgreSQL + pgvector + LLM + RAG + tool calling + workflow/agent logic + Docker + AWS.

GitHub: Every major project should have a strong README, architecture diagram, setup instructions, environment-variable documentation, screenshots/examples and API documentation.

Resume & Career Positioning

Suggested headline: Software Engineer | Python | Backend | GenAI | RAG | LLM Applications.

Career story: “I have enterprise production and support experience working with production systems, SQL, debugging, service-level issues and distributed/data platforms. I am building on that background with Python backend development, FastAPI, cloud and GenAI to build AI-powered production systems.”

Emphasize production troubleshooting, root-cause analysis, microservices, service diagnostics, SQL/data-platform exposure and the new AI-powered portfolio.

Final Priority Formula

40% Projects / coding.

25% Python / Backend / GenAI learning.

20% DSA.

10% Cloud / certification.

5% Career positioning / applications / revision.

During Weeks 11–12, shift more time toward DSA, system design, AI system design and mock interviews.

The Most Important Rule

Do not try to become a deep-learning expert, ML researcher, Kubernetes expert, distributed-systems expert, cloud architect and competitive programmer all within 90 days.

Your focused goal: Become employable as a Software / Backend Engineer who can build production-oriented AI applications.

Enterprise Experience + Python + FastAPI + PostgreSQL + Docker + AWS + LLMs + RAG + Tool Calling + AI System Design + DSA = AI-Powered Backend / GenAI Engineer.